

DB + 認証 フェーズ

DB + 認証 フェーズ

User テーブルと JWT + HttpOnly Cookie によるログイン基盤の構築

Hyonun Kin

2026-04-28

目的・背景

本サービスは社内15人規模の利用を想定しており、リクエスト毎に「誰がアクセスしているか」を識別できる必要がある。校正 API（次フェーズ）と管理者 API は認証必須のため、本フェーズで土台となる認証基盤を構築する。

設計判断としては JWT + HttpOnly Cookie を採用したが、これは**スケーラビリティのためではない**。15人規模ではセッション方式でも DB 負荷は問題にならず、JWT のスケール優位性は効かない。それでも JWT を選んだ理由は、

- 構造がシンプル（DB に session テーブルを持たなくて済む）
- 学習目的（exam で説明する題材として典型的）

の2点である。代わりに「ログアウトの即時無効化ができない」という構造的欠点を抱えるが、社内・低リスク環境のため許容する。

要件

機能要件：

- ID/パスワードでログインできる
- ログアウトで Cookie を破棄できる
- 認証必須エンドポイントは Cookie の JWT を検証してから処理する
- `user` / `admin` の2ロールを区別できる
- サーバー起動時に初期 admin を自動作成する（環境変数で投入、幕等）

非機能要件：

- パスワードは bcrypt でハッシュ化して保存
- JWT は HS256、有効期限 24 時間
- Cookie は HttpOnly + SameSite=Lax
- JWT_SECRET は環境変数から読み込み、コードにハードコードしない

設計

認証フロー



ログイン成功で Cookie がブラウザに保存され、以降のリクエストで自動付与される。サーバーは依存関数 `current_user` で Cookie を読み、`jwt.decode()` で検証する。

JWT クレーム構造

クレーム	型	内容
<code>sub</code>	<code>str</code>	<code>user.id</code> を文字列化したもの
<code>username</code>	<code>str</code>	表示用
<code>role</code>	<code>str</code>	<code>user</code> または <code>admin</code>
<code>exp</code>	<code>int</code>	発行から 24 時間後の UTC 秒

提供エンドポイント

- `POST /api/login` — 認証して Cookie を発行
- `POST /api/logout` — Cookie を空にして失効させる（発行済み JWT は `exp` まで有効）
- `GET /api/me` — 現在のログインユーザー情報（疎通確認用）

- GET /health — phase 1 から継続

Cookie の付与方針

- httponly=True — JavaScript からのアクセス禁止 → XSS 対策
- samesite="lax" — クロスサイトの POST に Cookie を付けない → CSRF 対策
- secure=False — 開発時は HTTP 経由のため。本番は環境差し替えで True にする想定

実装

ファイル構成

phase 2 で追加・変更したファイル：

```
backend/api/
├─ auth.py           # bcrypt ハッシュ、JWT 発行/検証、current_user 依存
├─ initial_admin.py  # 起動時の冪等 admin 作成
├─ models.py         # User テーブル定義
├─ session.py        # 非同期エンジン + SessionLocal + get_session
├─ settings.py       # pydantic-settings で env から読み込み
└─ app.py            # lifespan + /api/login /api/logout /api/me

backend/tests/
├─ conftest.py       # sqlite in-memory + httpx.AsyncClient
├─ test_auth.py      # ログイン成功/失敗、me、logout、無効化
├─ test_initial_admin.py # 冪等性、空パス時スキップ
└─ test_smoke.py     # /health
```

主要コード抜粋 (auth.py)

```
ALGORITHM = "HS256"
COOKIE_NAME = "token"

def hash_password(password: str) -> str:
    return bcrypt.hashpw(password.encode(), bcrypt.gensalt()).decode()

def verify_password(password: str, password_hash: str) -> bool:
    return bcrypt.checkpw(password.encode(), password_hash.encode())

def create_token(user: User) -> str:
    payload = {
        "sub": str(user.id),
        "username": user.username,
        "role": user.role,
        "exp": datetime.now(UTC) + timedelta(hours=settings.jwt_expire_hours),
    }
    return jwt.encode(payload, settings.jwt_secret, algorithm=ALGORITHM)

async def current_user(
    token: str | None = Cookie(default=None, alias=COOKIE_NAME),
    session: AsyncSession = Depends(get_session),
) -> User:
    if token is None:
        raise HTTPException(401, "not authenticated")
    try:
        payload = decode_token(token)
    except jwt.PyJWTError as e:
        raise HTTPException(401, "invalid token") from e
    user = await session.get(User, int(payload["sub"]))
    if user is None or not user.is_active or user.deleted_at is not None:
        raise HTTPException(401, "user not found")
    return user
```

`current_user` は FastAPI の `Depends` で各エンドポイントに差し込めば、認証必須ルートは依存を1行追加するだけで保護できる。`require_admin` も同形で `role` を弾く。

初期 admin の冪等作成

```
async def ensure_initial_admin(session: AsyncSession) -> None:
    if not settings.initial_admin_password:
        return # 環境変数が空なら何もしない（ローカル/テスト用）
    existing = (
        await session.execute(
            select(User).where(User.username == settings.initial_admin_username)
        )
        ).scalar_one_or_none()
    if existing is not None:
        return # 既にある → 何もしない
    session.add(
        User(
            username=settings.initial_admin_username,
            password_hash=hash_password(settings.initial_admin_password),
            role="admin",
            is_active=True,
        )
    )
    await session.commit()
```

`app.lifespan` で `Base.metadata.create_all` の直後に呼ぶ。サーバーを何度再起動しても重複エラーは出ない。

テスト用 DB の扱い

本番は PostgreSQL (asyncpg)、テストは sqlite in-memory を使う。`conftest.py` で `DATABASE_URL` 環境変数を `sqlite+aiosqlite:///memory:` に上書きしてから `api` 配下をインポートする。

sqlite の in-memory はコネクションごとに別インスタンスになるため、`session.py` で `StaticPool` を使い1コネクションを共有する設定にしている。

検証

項目	結果
<code>uv run pytest -v</code>	11 件 pass、warning なし
<code>test_login_success_sets_cookie</code>	OK
<code>test_login_wrong_password</code>	401
<code>test_login_unknown_user</code>	401
<code>test_me_requires_cookie</code>	401
<code>test_me_with_cookie_returns_user</code>	OK
<code>test_logout_clears_cookie</code>	logout 後 /me が 401
<code>test_inactive_user_cannot_login</code>	is_active=False で 401
<code>test_creates_admin_when_password_set</code>	admin 作成される
<code>test_idempotent_does_not_duplicate</code>	3 回呼んで 1 件のみ
<code>test_skips_when_password_empty</code>	何も作られない

確認観点 (architecture.md より)

- 設計通り (user テーブル、JWT 24h、HttpOnly Cookie、SameSite=Lax)
- 環境変数をハードコードしてない (`.env.example` のみ、JWT_SECRET 等は実値空)
- エラーハンドリング (401, 403, 各種フォールバック) が入っている
- 自分で説明できるコード (依存関数 `current_user` だけ理解すれば全エンドポイントが守られる仕組み)

次フェーズへの引き継ぎ

phase 3 (校正 API) で着手するもの :

- `api/ai.py` — Gemini 呼び出し (LangChain 経由)

- `prompts/correct.yaml` – 校正プロンプト (system + user + structured output)
- `POST /api/correct` – `current_user` を依存に追加し、認証必須にする
- 結果は **DB に保存しない** (architecture.md の方針に従う)
- 異常系: Gemini の API エラー、レート制限、ネットワーク切断のハンドリング

注意点:

- JWT の有効期限は 24 時間。長時間放置されたタブで失効するケースをフロント側でハンドリング (401 を受けたら /login へリダイレクト)。
- ログアウト後に古い JWT を持ち回されたら exp まで有効になることを許容している。社内・低リスクのため。